ChatGPT Share

what is GARF_RB01

I'm checking whether **GARF_RB01** is a product code, model name, or internal identifier so I can pin down the right meaning.

Thought for 8s

GARF_RB01 appears to be an **RFID tag/card model or internal product code**, not a widely documented public standard name.

From the available public search results, I could **not verify an official manufacturer datasheet or catalog entry** for `GARF_RB01` itself. What *is* consistent from the surrounding RFID context is that it likely refers to a **13.56 MHz RFID/NFC credential** such as a card, key fob, or access token, because common products in this category use similar naming and operate with standards like **ISO/IEC 14443 Type A**. Adafruit +1

So the safest interpretation is:

- **GARF_RB01** = probably a **vendor-specific RFID product/model identifier**
- **Not** a general RFID protocol name
- It may be tied to a specific batch, card family, or internal inventory label rather than a public-facing model

If you saw it written together with something like **GARFRB0125310144** or **N6001000250506C0337**, those are very likely:

- a longer **part number**
- a **batch/serial/manufacturing code**
- or a **UID-related/internal tracking code**

I do **not** see enough reliable public evidence to claim the exact manufacturer or exact chip type yet.

What it most likely is

A practical description would be:

> **GARF_RB01 is likely an RFID credential/card identifier used for access control or RFID system integration.**

What would identify it exactly

To determine it precisely, one of these is needed:

- a photo of the card or label
- the printed frequency, such as **13.56 MHz** or **UHF 860–960 MHz**
- the reader model it works with
- a datasheet or packaging label

If you want, I can next help you determine whether **GARF_RB01 is HF/NFC or UHF**, and infer its likely protocol family from the codes you already have.

Sources

okay, please

I'm narrowing it down by checking whether the code shows up with HF/NFC cards or with UHF EPC tags, then I'll map that to the likely protocol family.

Thought for 18s

Based on the code you gave earlier, **the strongest guess is that GARF_RB01 is a UHF RFID tag/label, not an HF/NFC card**.

Why I think that

The biggest clue is this part of the longer code:

```
N60010002505O6C0337
```

That "**N600**" pattern lines up much more closely with **NXP UCODE / RAIN RFID UHF** naming seen in the market than with common 13.56 MHz NFC families like MIFARE, NTAG, or DESFire. NXP's UCODE line is a **UHF / RAIN RFID** family, and RAIN RFID operates in the **860–960 MHz UHF band** using the **EPC Gen2 / ISO/IEC 18000-63** air interface.

NXP Semicondu... +2

Most likely protocol family

So the most likely protocol stack is:

- **Technology:** UHF RFID / RAIN RFID

- **Frequency:** roughly **860–960 MHz** depending on region
- **Protocol / standard: EPC Class 1 Gen 2**, standardized as **ISO/IEC 18000-63**

NXP Semicondu... +1

What this means in practice

If that inference is right, **GARF_RB01** is probably:

- a **vendor-specific tag model name**
- using an **NXP UCODE-family chip** or something very similar
- intended for **longer-range identification** such as logistics, asset tracking, parking, or access/vehicle tagging rather than tap-to-read NFC use cases NXP Semicondu... +2

What I am *not* fully sure about

I could **not find a public authoritative datasheet or catalog page** for the exact string **GARF_RB01** or the full code **GARFRB0125310144**. So I cannot honestly say "this exact product is definitely model X from manufacturer Y." The conclusion above is an **evidence-based inference**, mainly from the **N600** clue and the way UCODE/RAIN RFID products are named in the market. NXP Semicondu... +1

Best current conclusion

Most likely:
GARF_RB01 = a vendor/internal model code for a passive UHF RAIN RFID tag, probably using an NXP UCODE-family chip, and likely speaking EPC Gen2 / ISO/IEC 18000-63.

NXP Semicondu... +2

If you want, I can next give you a **datasheet-style inferred specification sheet** for GARF_RB01 with fields like frequency, memory, read range, protocol, and likely applications.

Sources

sure, please

I'm checking a few manufacturer and distributor traces so the spec sheet is grounded in the closest match I can verify, then I'll label clearly what is confirmed versus inferred.

Thought for 10s

Below is an **inferred datasheet-style summary** for **GARF_RB01** based on the code pattern you provided and the closest verifiable public RFID references.

GARF_RB01 — Inferred Specification Sheet

Item	Value
Product name	**GARF_RB01**
Product type	Likely **passive UHF RFID tag / label / credential**
Likely chip family	Likely **NXP UCODE / RAIN RFID** family
RFID band	Likely **UHF 860–960 MHz**
Likely air interface	**EPC Class 1 Gen2 / Gen2v2**, standardized under **ISO/IEC 18000-63**
Power source	**Passive** (powered by reader field)
Typical use cases	Asset tracking, logistics, vehicle identification, access tagging, inventory control
Read range	**Application-dependent**; can vary widely by antenna, tag construction, mounting surface, reader power, and region
Public documentation status	**No exact public datasheet found** for `GARF_RB01` itself

What is reasonably supported

NXP's **UCODE** line is a **RAIN RFID UHF** product family, intended for item identification, logistics, automotive, and similar tracking use cases. NXP also states that UCODE devices are compliant with the **EPC Gen2** ecosystem used in UHF RAIN RFID. NXP Semicondu... +2

That means the most plausible technical baseline for a code like **GARF_RB01** is:

- **UHF / RAIN RFID**
- **860–960 MHz**
- **EPC Gen2 / ISO/IEC 18000-63**
- **Passive tag behavior** rather than NFC-style tap communication. NXP Semicondu... +2

Important caution

I could **not find an official manufacturer page or datasheet** for the exact strings:

- `GARF_RB01`
- `GARFRB0125310144`
- `N6001000250506C0337`

So the sheet above is **not a confirmed manufacturer datasheet**. It is the **best-fit inferred spec** from the surrounding identifier pattern and the closest public references to NXP UCODE / RAIN RFID products. NXP Semicondu... +1

Likely protocol stack

If this inference is correct, the protocol stack would look like this:

- **Radio technology:** UHF RFID / RAIN RFID
- **Air interface:** EPCglobal Class 1 Gen 2 / Gen2v2
- **International standard mapping:** ISO/IEC 18000-63. NXP Semicondu... +1

Likely memory model

I cannot confirm the exact chip, so I cannot honestly state the exact memory sizes for GARF_RB01. However, many public UHF tags based on NXP UCODE-family chips commonly expose fields such as:

- **EPC memory**
- **TID memory**
- optional **User memory**

For example, one public UHF label using an NXP UCODE chip lists **EPC 128 bits** and **TID 96 bits**, which shows the general kind of memory structure common in this product class. That should be treated as an **example**, not as the confirmed GARF_RB01 configuration.

atlasRFIDstore

Likely electrical / operating profile

These are the most defensible inferred characteristics:

- **Reader-powered passive operation**
- **Longer read distance than HF/NFC systems**
- performance strongly affected by:
  - antenna design
  - tag size
  - mounting material
  - polarization
  - regional frequency rules
  - reader output power. NXP Semicondu... +2

Practical identification guide

You can verify whether GARF_RB01 is really this kind of tag by checking:

1. **Does it work with a UHF reader** instead of a 13.56 MHz NFC reader?
2. **Does the reader mention EPC / TID / Gen2 / 18000-63?**
3. **Is the operating band shown as 860–960 MHz or 902–928 MHz?**
4. **Is the read distance more than a few centimeters?** If yes, that strongly points to UHF rather than HF/NFC.

Best current one-line description

GARF_RB01 is most likely a vendor-specific passive UHF RAIN RFID tag identifier, probably in the NXP UCODE ecosystem and likely using EPC Gen2 / ISO/IEC 18000-63.

NXP Semicondu... +2

I can also make this into a cleaner **formal datasheet page** with sections like **Features, Electrical Characteristics, Memory Map, Protocol, Applications, and Ordering Information**.

Sources

give me more detailed information for ISO/IEC 14443

I'm pulling together a standards-focused explanation so I can separate the protocol layers, tag types, anti-collision, and where common cards like MIFARE fit.

is it possible to implement the ISO/IEC 14443 with STM32 series ARM cpu?

I'm checking the practical hardware path here, because the key question is not just the CPU but whether you also need an NFC/RFID analog front end.

Thought for 38s

Yes — **it is possible to build an ISO/IEC 14443 system with an STM32-series ARM MCU**, but **not with the STM32 alone** in the usual practical sense. You normally pair the STM32 with a **13.56 MHz NFC/RFID front-end IC** such as the **ST25R3916/ST25R3916B**, which

handles the RF analog part plus much of the frame coding/decoding for **ISO/IEC 14443 Type A and Type B**. ST explicitly documents STM32 Nucleo expansion boards based on these reader chips and states that the ST25R3916/3916B supports ISO/IEC 14443 A/B.

STMicroelectron... +3

The practical answer

What STM32 does

The **STM32** is the main controller. It typically handles:

- application logic
- command processing
- higher protocol layers
- SPI/I2C/UART communication with the NFC front-end
- UI, networking, storage, security, and system control

What the NFC front-end does

The **RFID/NFC transceiver/front-end** handles the hard radio work:

- generating the **13.56 MHz** carrier
- ASK/load modulation handling
- receive analog front-end
- frame coding/decoding
- timing-sensitive ISO14443 operations
- anti-collision support in reader mode, depending on the stack/device

That is exactly why ST offers combinations like:

- **STM32 + X-NUCLEO-NFC06A1 / NFC08A1**
- with **ST25R3916 / ST25R3916B**
- plus the **RFAL / X-CUBE-NFC** software stack for STM32 STMicroelectron... +4

So can STM32 implement ISO/IEC 14443 by itself?

In theory

At a very experimental level, you could try to generate and sample signals with MCU peripherals plus custom analog circuitry.

In practice

For a real product, **no, not STM32 alone**. ISO/IEC 14443 is a **contactless smart-card RF standard** at **13.56 MHz** with strict analog, modulation, timing, field strength, and

interoperability requirements. ST's documentation around the ST25R39xx family exists precisely because a dedicated RF front-end is the practical way to implement it correctly. ST also notes that interoperability testing for ISO 14443 requires proper reference PICCs and conformance-style measurements. STMicroelectron... +2

Recommended architecture

A typical implementation looks like this:

```
[STM32 MCU]
    |
   SPI
    |
[ST25R3916 / ST25R3916B]
    |
 matching network
    |
 13.56 MHz antenna
    |
[ISO/IEC 14443 card / tag]
```

This is the normal and recommended path for an STM32-based design. ST's Nucleo expansion boards and software ecosystem are built around exactly this architecture.

STMicroelectron... +3

Reader vs card emulation

There are two very different goals:

1. Reader / poller

This is the easier and more common STM32 use case. The system reads cards such as:

- ISO/IEC 14443 Type A cards
- ISO/IEC 14443 Type B cards
- NFC Forum tags, depending on the front-end and software stack

ST's documentation explicitly says the ST25R3916/3916B supports **reader mode** for ISO/IEC 14443 A/B and other HF/NFC standards. STMicroelectron... +3

2. Card emulation / listener / PICC behavior

This is more specialized. Some ST documentation and software references include **NFC-A listener / passive target helpers**, but support level depends on the exact chip and software stack. So this is possible in some cases, but it is more constrained than reader

mode and should be verified against the exact front-end and firmware stack you choose. STMicroelectron... +2

Best ST parts to use with STM32

For an STM32-based ISO/IEC 14443 project, the most relevant ST choices from current public materials are:

- **ST25R3916**
- **ST25R3916B**
- STM32 Nucleo expansion boards such as **X-NUCLEO-NFC06A1** and **X-NUCLEO-NFC08A1**

ST describes these as NFC card-reader boards for STM32 and states support for **ISO14443A/B**. STMicroelectron... +3

Software stack you would use

A realistic software stack is:

- **STM32Cube**
- **X-CUBE-NFC6** or the relevant ST NFC package
- **RFAL** (RF/NFC abstraction layer)
- your application code on top

ST's RFAL documentation mentions a **Poller (ISO14443A PCD)** interface and helper support around NFC-A listener functionality. STMicroelectron... +1

Engineering points that matter

Even with the right STM32 and RF front-end, the hard part is often not the MCU firmware. It is:

- antenna design
- matching network tuning
- EMC/noise control
- power delivery to the RF section
- conformance and interoperability testing

ST's antenna-design application note emphasizes that correct ISO 14443 operation requires standards-based RF measurements and reference PICCs for testing. STMicroelectron...

Bottom line

Yes, STM32 can absolutely be used in an ISO/IEC 14443 system.
But the practical product architecture is:

STM32 MCU + dedicated NFC/RFID front-end IC + 13.56 MHz antenna

—not STM32 by itself. STMicroelectron... +3

Good starting point

If you want the fastest path, start with:

- an **STM32 Nucleo**
- **X-NUCLEO-NFC08A1** or similar
- **ST25R3916B**
- ST's **RFAL / X-CUBE-NFC** software stack STMicroelectron... +2

I can also give you a **concrete example architecture for STM32 + ST25R3916B**, including SPI connection, firmware layers, and which ISO/IEC 14443 functions live in which chip.

Sources

sure, please

I'm putting this into a hardware-and-firmware breakdown so you can see exactly what the STM32 handles, what the RF chip handles, and how the signal flow works end to end.

Thought for 14s

Yes — here is a **concrete STM32 + ISO/IEC 14443 architecture example**.

Recommended architecture

```
ISO/IEC 14443 Card / PICC
        ^
        | 13.56 MHz RF field
        v
+-----------------------------+
|   Antenna + Matching Network |
+-----------------------------+
               |
               v
+-----------------------------+
|  ST25R3916B NFC/RFID Front End |
|  - RF field generation       |
|  - ASK/load modulation       |
|  - RX analog front end       |
```

```
|  - frame coding/decoding     |
|  - low-level timing support  |
+------------------------------+
                |
          SPI / IRQ / GPIO
                |
                v
```

This is the normal product structure ST documents for STM32-based NFC reader systems. The **X-NUCLEO-NFC08A1** expansion board is built around the **ST25R3916B** and is specifically intended to be used with **STM32 Nucleo** boards. STMicroelectron... +1

What each chip does

1) STM32 MCU responsibilities

The STM32 side typically handles:

- application state machine
- user logic
- access-control/business logic
- APDU or higher-level command handling
- host communication
- storage, UI, networking, security
- control of the RF chip over SPI and interrupts

ST's software package for STM32, **X-CUBE-NFC6**, provides the firmware framework that sits on the MCU side for this type of design. STMicroelectron... +1

2) ST25R3916B responsibilities

The **ST25R3916B** is the dedicated NFC/RFID front end. ST describes it as a high-performance universal NFC device supporting reader and card-emulation-related modes when applicable, and the associated STM32 expansion-board documentation states that it manages **frame coding and decoding** for standards such as **ISO/IEC 14443 Type A and Type B**. STMicroelectron... +2

In practice, the ST25R3916B handles:

- 13.56 MHz carrier generation
- analog transmit/receive path
- modulation/demodulation
- low-level protocol timing assistance
- parts of the framing work needed for ISO/IEC 14443 reader operation

That is why this dedicated IC is used instead of trying to do the RF job directly from a general MCU. STMicroelectron... +1

Where ISO/IEC 14443 layers live

A useful way to think about it is this:

RF / physical layer

Mostly in the **ST25R3916B** plus the antenna and matching network. ST's docs and antenna-design app note emphasize that analog RF behavior, matching, and standards-based measurements are crucial for ISO 14443 interoperability. STMicroelectron... +1

Framing / low-level reader operations

Shared, but heavily assisted by the **ST25R3916B**. ST says the chip manages frame coding and decoding in reader mode for standard NFC/HF RFID applications including ISO14443A/B. STMicroelectron... +1

Protocol handling above that

Handled in firmware on the **STM32** through ST's software stack. The RFAL documentation states that its **ISO-DEP** module establishes the **ISO14443-4** protocol link and handles activation, deactivation, timings, and data exchange in poller and listener mode.

STMicroelectron...

Example firmware stack on STM32

A practical firmware stack looks like this:

```
Application
  └── Card handling logic / business rules
      └── ISO-DEP / APDU exchange layer
          └── RFAL protocol layer
              └── ST25R3916B driver
                  └── SPI + IRQ + GPIO HAL
                      └── STM32Cube HAL / LL
```

ST's **RFAL** and **X-CUBE-NFC6** are the key software building blocks for this structure on STM32. The RFAL manual specifically calls out support for **ISO-DEP (ISO14443-4)** protocol establishment and data exchange. STMicroelectron... +1

Reader flow example

Here is a simplified **reader / poller** sequence for ISO/IEC 14443:

1. **STM32 initializes ST25R3916B**

2. ST25R3916B turns on the RF field
3. Polling for **Type A** or **Type B**
4. Detect card response
5. Anti-collision and select
6. Activate protocol
7. If needed, establish **ISO-DEP** session
8. Exchange application commands
9. Halt / deselect / stop polling

The exact low-level timing and framing are assisted by the ST25R3916B, while the session logic and application behavior run on the STM32. ST's RFAL documentation explicitly mentions ISO-DEP link establishment and data exchange, and ST's expansion-board documents state support for ISO14443A/B operation. STMicroelectron... +1

Hardware signals between STM32 and ST25R3916B

A minimal interface usually includes:

- **SPI SCK**
- **SPI MOSI**
- **SPI MISO**
- **CS/NSS**
- **IRQ** from ST25R3916B to STM32
- **RESET** or enable pin
- power rails and ground

The board-level realization is shown in ST's X-NUCLEO-NFC08A1 hardware documents and user manual for STM32 Nucleo integration. STMicroelectron... +1

Antenna and analog section

This part is critical.

Even when the digital design is fine, ISO/IEC 14443 projects often fail because of:

- poor antenna geometry
- bad matching/tuning
- incorrect Q factor
- noisy power rails
- layout issues near the RF section
- lack of standards-based validation

ST's antenna-design application note says measurements must be performed according to the required standard and specifically notes ISO 14443 testing against the reference PICC

defined in **ISO/IEC 10373-6**. STMicroelectron...

Can STM32 do card emulation too?

Potentially yes, depending on the exact mode and stack, but **reader mode is the most straightforward starting point**. ST's ST25R3916B product page and the X-NUCLEO-NFC08A1 manual indicate support for **reader/writer** and **card emulation** modes, while the RFAL documentation mentions activation/deactivation in both **Poller** and **Listener** mode for ISO-DEP. STMicroelectron... +2

Best starting hardware

A very practical starting setup is:

- **STM32 Nucleo board**
- **X-NUCLEO-NFC08A1**
- **X-CUBE-NFC6**
- ST example firmware

That combination is directly documented by ST and avoids a lot of early analog and layout mistakes while you learn the stack. STMicroelectron... +2

Best starting project plan

Stage 1

Bring up the Nucleo + X-NUCLEO-NFC08A1 demo and detect a known ISO14443A card.

STMicroelectron... +1

Stage 2

Trace the firmware path:

- board init
- ST25R3916B driver init
- RFAL poller init
- card detect/select
- higher-layer data exchange. STMicroelectron... +1

Stage 3

Move to your custom application:

- access control
- UID reading
- Type 4 tag interaction
- APDU exchange if needed. STMicroelectron...

Stage 4

Only then move to a custom PCB with your own antenna and matching network, using ST's antenna-design guidance. STMicroelectron...

Bottom line

For **ISO/IEC 14443 with STM32**, the clean engineering split is:

- **STM32** = protocol control, firmware, application
- **ST25R3916B** = RF/NFC analog front end and low-level framing support
- **antenna + matching network** = critical RF hardware

That is the standard, practical way to implement it. STMicroelectron... +2

I can next give you either a **pin-level STM32 ↔ ST25R3916B connection example** or a **step-by-step ISO14443A firmware flow**.

Sources

sure, please

I'm getting the exact ST reference material for the interface pins and the protocol flow so I can keep it concrete instead of hand-wavy.

Thought for 9s

Absolutely. Here are both: a **pin-level connection example** and a **step-by-step ISO/IEC 14443A reader flow** for **STM32 + ST25R3916B**.

1) Pin-level connection example

This is the **typical control interface** between the STM32 and the NFC front end. The **ST25R3916B** is designed to be hosted by a controller over **SPI**, and ST's X-NUCLEO-NFC08A1 board is the reference example for using it with STM32 Nucleo boards.

STMicroelectron... +1

```
STM32                          ST25R3916B
-----                          ----------
3V3    ----------------------> VDD
GND    ----------------------> GND
```

```
SPI_SCK  ------------------> SCLK
SPI_MOSI ------------------> MOSI
SPI_MISO <------------------ MISO
GPIO_CS  ------------------> NSS / CS

GPIO_IRQ <------------------ IRQ output from ST25R3916B
GPIO_RST ------------------> EN / RESET control (board-dependent)

Optional:
GPIO field detect / status <- additional status lines if your board exposes
```

Minimum signals you should plan for

At minimum, you should allocate:

- **SPI clock**
- **SPI MOSI**
- **SPI MISO**
- **chip select**
- **one interrupt input**
- **one reset/enable output**
- stable **3.3 V** and clean **ground**

That matches the normal host-controller pattern used by ST's reader boards and RFAL software stack. STMicroelectron... +1

2) RF section you still need

Even with the STM32 and ST25R3916B connected, you do **not** yet have a complete ISO/IEC 14443 reader unless you also add:

- a **13.56 MHz antenna**
- a **matching network**
- layout and tuning appropriate for ISO 14443 operation

ST's antenna-design guidance is clear that RF performance and interoperability depend heavily on the analog design, not just the firmware. STMicroelectron... +2

A simplified hardware chain looks like this:

```
STM32  <SPI/IRQ>  ST25R3916B  <RF pins>  matching network  <->  13.56 MHz a
```

3) Firmware layering on STM32

A practical firmware stack on the STM32 side usually looks like this:

```
Your application
  └─ card handling / business logic
      └─ ISO-DEP / APDU layer (if needed)
          └─ RFAL NFC-A / NFC-B modules
              └─ ST25R3916B device driver
                  └─ STM32 HAL / SPI / GPIO / IRQ
```

ST's **X-CUBE-NFC6** package and **RFAL** are specifically intended for this architecture and support **NFC-A / ISO14443A**, **NFC-B / ISO14443B**, and **ISO-DEP (ISO14443-4)** on STM32-hosted ST25R devices. STMicroelectron... +1

4) Step-by-step ISO/IEC 14443A reader flow

Here is the practical reader flow for a Type A card.

Stage A — bring up the hardware

1. Power the STM32 and ST25R3916B.
2. Initialize SPI, IRQ GPIO, reset GPIO.
3. Reset and initialize the ST25R3916B.
4. Configure the RF front end for **NFC-A / ISO14443A poller mode**.

RFAL explicitly provides an **NFC-A Poller (ISO14443A PCD)** interface and handles technology detection and activation support. STMicroelectron...

Stage B — start RF polling

5. Turn on the RF field.
6. Transmit the **REQA** or **WUPA** request.
7. Listen for an **ATQA** response from a card.

This is the initial presence-detection phase for ISO/IEC 14443A, and RFAL's NFC-A module is built to handle this technology-detection step. STMicroelectron...

Stage C — anti-collision and selection

8. Run anti-collision.
9. Read the card UID parts.
10. Perform **SELECT**.
11. Receive **SAK** and determine the selected card type / cascade completion.

RFAL states that it supports **anticollision/collision resolution**, **card selection/activation**, and helper methods up to the **ISO14443-3** layer. STMicroelectron...

Stage D — activate higher-layer communication

12. If the card supports it, continue into **ISO-DEP / ISO14443-4** activation.
13. Exchange higher-layer data, such as APDUs for Type 4 style applications.

ST's RFAL documentation states that the **ISO-DEP** module establishes the **ISO14443-4** protocol link and handles data exchange after activation. STMicroelectron... +1

Stage E — end the session

14. Halt, deselect, or deactivate the card.
15. Continue polling for the next card.

RFAL documentation also calls out support for **sleep/deactivation** as part of the NFC-A flow. STMicroelectron...

5) Simplified sequence diagram

```
STM32/RFAL           ST25R3916B               Card (PICC)
-----------          ----------               -----------
init()     --------> configure chip
field on   -------> generate RF field
REQA       -------> transmit RF frame ------> receives REQA
ATQA       <------- receive/decode  <------- sends ATQA
anticoll.  -------> transmit commands ------> UID response
UID/SAK    <------- receive/decode  <------- selection response
RATS*      -------> if ISO-DEP needed ------> protocol activation
ATS*       <------- receive/decode  <------- ATS
APDU data* <-----> framed exchange  <-----> app commands
HLTA/deact -------> stop session
```

* = only when the card/application uses **ISO-DEP / ISO14443-4**. RFAL supports both the NFC-A lower layers and ISO-DEP higher-layer link establishment. STMicroelectron... +1

6) What lives where

In the ST25R3916B

The ST25R3916B provides the **analog front end** and a **highly integrated data framing system** for NFC-A/B and related technologies. That is the low-level RF-heavy part you do not want to implement directly on a general MCU. STMicroelectron...

In the STM32

The STM32 runs:

- RFAL
- your state machine
- card handling logic

- ISO-DEP and APDU orchestration
- application behavior such as access control, payments, logging, UI, or networking.

STMicroelectron... +1

7) Best development path

The easiest way to start is:

- **STM32 Nucleo board**
- **X-NUCLEO-NFC08A1**
- **X-CUBE-NFC6 / RFAL examples**

That gives you a known-good reference design for both hardware and firmware before you attempt a custom PCB. ST positions these tools specifically for evaluating and developing STM32-based NFC reader systems. STMicroelectron... +2

8) Practical advice

For a first project, do this in order:

1. Get the ST demo board working.
2. Read a known ISO14443A card UID.
3. Trace the RFAL polling flow in code.
4. Add your own application logic.
5. Only then design a custom antenna and PCB.

That order reduces risk because most early failures in ISO/IEC 14443 projects are in the RF section and board tuning, not in the Cortex-M application code. STMicroelectron... +1

If you want, next I can give you a **sample STM32 firmware skeleton** for the polling loop using RFAL.

Sources

okay, please

I'm putting together a minimal but realistic STM32 polling skeleton based on ST's RFAL stack, so it matches the actual software layering instead of pseudocode.

Thought for 15s

Below is a **minimal STM32 firmware skeleton** for an **ISO/IEC 14443A poller/reader** using **STM32 + ST25R3916B + X-CUBE-NFC6/RFAL**. ST says X-CUBE-NFC6 provides the middleware for STM32 to control the ST25R3916/ST25R3916B, and RFAL includes **NFC-A / ISO14443A**, **NFC-B / ISO14443B**, and **ISO-DEP (ISO14443-4)** support. ST also says the package includes sample applications such as **Tag Detect/Card Emulation** and **Read/Write NDEF**. STMicroelectron... +2

Firmware structure

A practical stack looks like this:

```
main()
 ├─ board_init()
 ├─ spi_gpio_irq_init()
 ├─ st25r3916b_init()
 ├─ rfal_init()
 └─ main_loop()
     ├─ rfal_worker()
     ├─ poll_for_type_a_card()
     ├─ select_card()
     ├─ activate_iso_dep_if_needed()
     └─ exchange_application_data()
```

That split matches ST's documented architecture: **STM32Cube HAL/BSP**, **RFAL middleware**, and application code on top. RFAL is organized around technology and protocol modules, and its NFC module provides common poller/listener activities.

STMicroelectron... +1

Minimal C skeleton

C

```c
#include "main.h"
#include "rfal_api.h"          // placeholder include name; use your package's
#include "st25r3916b_driver.h" // placeholder include name; use your package's

typedef enum {
    APP_IDLE = 0,
    APP_POLL,
    APP_CARD_FOUND,
    APP_CARD_ACTIVE,
    APP_ERROR
} app_state_t;

static app_state_t g_state = APP_IDLE;
```

```
static void board_init(void);
static void nfc_hw_init(void);
static bool nfc_stack_init(void);
static bool poll_type_a_card(void);
static bool activate_selected_card(void);
static void process_active_card(void);
static void error_recovery(void);
```

How to read this skeleton

The code above is a **structure template**, not a copy-paste final build. The exact RFAL symbol names and header names can vary by package version and board example, but the flow is aligned to ST's documented model:

- RFAL must be configured by the user via platform integration files. STMicroelectron...
- RFAL provides **NFC-A poller** support at the technology layer. STMicroelectron... +1
- RFAL supports **anticollision/collision resolution**, **selection/activation**, and **ISO-DEP** for higher-layer exchanges. STMicroelectron...

The ISO/IEC 14443A flow this skeleton is following

For **Type A**, the logical flow is:

1. Initialize STM32 peripherals and the ST25R3916B front end.
2. Initialize RFAL and configure **poller mode**.
3. Turn on the RF field.
4. Perform **technology detection** for NFC-A / ISO14443A.
5. Run **anticollision** and **card selection**.
6. If needed, activate **ISO-DEP (ISO14443-4)**.
7. Exchange application commands.
8. Deselect, halt, or resume polling. STMicroelectron... +1

What you should implement first

For a first working project, keep it simple:

Version 1

Read only:

- card present or not
- UID
- ATQA / SAK if useful for diagnostics

Version 2

Add:

- ISO-DEP activation
- APDU exchange
- basic application command handling

Version 3

Add:

- retry handling
- timeout/error recovery
- hot-removal detection
- multiple-card edge cases

That progression fits ST's sample-app approach, where the package already includes example projects for tag detection and NDEF operations. STMicroelectron... +1

Recommended starting point

The fastest path is to start from:

- **STM32 Nucleo**
- **X-NUCLEO-NFC08A1**
- **X-CUBE-NFC6** sample project

ST says those samples are provided for supported Nucleo targets and are intended as ready-to-build starting points. STMicroelectron... +1

Important caution

Two parts usually need adaptation before this kind of skeleton will compile cleanly in your project:

- the **platform layer** for SPI, IRQ, delays, timing, and critical sections
- the exact **RFAL API names / sample wrappers** used by your package version and board example

ST's RFAL manual explicitly says the user must provide platform-specific settings through the RFAL platform layer. STMicroelectron...

I can give you the next step as a **more concrete STM32CubeIDE project skeleton with file-by-file structure**.

Sources

okay, please

I'm turning that into a file-by-file STM32CubeIDE layout so you can see where HAL, RFAL, the ST25R driver, and your app logic should live.

Thought for 8s

Here is a **practical STM32CubeIDE project skeleton** for an **ISO/IEC 14443A reader** using **STM32 + ST25R3916B + X-CUBE-NFC6/RFAL**.

ST's official stack for this is:

- **X-CUBE-NFC6** as the STM32 software package for **ST25R3916/ST25R3916B**. STMicroelectron... +1
- **X-NUCLEO-NFC08A1** as the reference reader board based on **ST25R3916B**. STMicroelectron... +1
- **RFAL** as the abstraction/protocol layer, including **NFC-A / ISO14443A**, **NFC-B**, and **ISO-DEP (ISO14443-4)** support. STMicroelectron... +1

Recommended project layout

```
MyNfcReader/
├─ Core/
│  ├─ Inc/
│  │  ├─ main.h
│  │  ├─ app_nfc.h
│  │  ├─ nfc_platform.h
│  │  ├─ nfc_reader.h
│  │  ├─ nfc_iso14443a.h
│  │  ├─ debug_log.h
│  │  └─ board_pins.h
│  └─ Src/
│     ├─ main.c
│     ├─ app_nfc.c
│     ├─ nfc_platform.c
│     ├─ nfc_reader.c
│     ├─ nfc_iso14443a.c
│     ├─ debug_log.c
```

```
│     └─ stm32xx_it.c
│
├─ Drivers/
│  └─ CMSIS/
```

This structure follows the same separation ST documents: **HAL/BSP**, **device driver**, **RFAL/middleware**, then **application code** on top. X-CUBE-NFC6 includes component drivers, board support, and sample applications for the X-NUCLEO-NFC06A1/X-NUCLEO-NFC08A1 boards. STMicroelectron... +1

What each folder should contain

`Core/`

This is your STM32Cube-generated application layer.

`main.c`

Use it only for:

- HAL init
- clock setup
- peripheral init
- calling `app_nfc_init()`
- calling `app_nfc_process()` in the loop

Keep `main.c` thin.

`app_nfc.c`

This should be your top-level NFC app orchestrator:

- initialize RFAL
- initialize the ST25R3916B path
- run the polling loop
- dispatch card events to application logic

`nfc_platform.c`

This file adapts RFAL to your STM32 platform:

- SPI read/write
- IRQ handling hooks
- delays/timers
- critical-section helpers

- millisecond/microsecond tick functions

RFAL requires user/platform integration and is meant to sit on top of STM32Cube HAL or equivalent platform code. STMicroelectron... +1

`nfc_reader.c`

This should contain the generic reader state machine:

- field on
- poll
- card detect
- select
- activate
- data exchange
- deactivate

`nfc_iso14443a.c`

This should contain the **Type A specific** flow:

- REQA / WUPA detection
- ATQA handling
- anti-collision
- UID assembly
- SAK interpretation
- optional ISO-DEP activation

RFAL provides **NFC-A Poller** support and collision resolution/card selection support up to the **ISO14443-3** layer. STMicroelectron...

Recommended file responsibilities

`app_nfc.h`

c

```c
#pragma once

#include <stdbool.h>
#include <stdint.h>
```

```c
bool app_nfc_init(void);
void app_nfc_process(void);
```

nfc_reader.h

```c
#pragma once

#include <stdbool.h>
#include <stdint.h>

typedef enum {
    NFC_READER_IDLE = 0,
    NFC_READER_FIELD_ON,
    NFC_READER_POLL_TECH,
    NFC_READER_RESOLVE_COLLISION,
    NFC_READER_SELECT,
    NFC_READER_ACTIVATE,
    NFC_READER_EXCHANGE,
    NFC_READER_DEACTIVATE,
    NFC_READER_ERROR
} nfc_reader_state_t;

bool nfc_reader_init(void);
void nfc_reader_process(void);
```

nfc_iso14443a.h

```c
#pragma once

#include <stdbool.h>
#include <stdint.h>

typedef struct {
    uint8_t uid[10];
    uint8_t uid_len;
    uint8_t sak;
    uint8_t atqa[2];
    bool    iso_dep_supported;
} iso14443a_card_t;

bool iso14443a_poll(iso14443a_card_t *card);
```

```
bool iso14443a_activate(iso14443a_card_t *card);
bool iso14443a_deactivate(void);
```

Minimal `main.c`

```c
#include "main.h"
#include "app_nfc.h"

int main(void)
{
    HAL_Init();
    SystemClock_Config();

    MX_GPIO_Init();
    MX_SPI1_Init();
    MX_USART2_UART_Init();   // optional for debug

    if (!app_nfc_init()) {
        Error_Handler();
    }

    while (1) {
        app_nfc_process();
    }
}
```

Minimal `app_nfc.c`

```c
#include "app_nfc.h"
#include "nfc_reader.h"

bool app_nfc_init(void)
{
    return nfc_reader_init();
}

void app_nfc_process(void)
{
```

```
    nfc_reader_process();
}
```

Minimal `nfc_reader.c`

C

```c
#include "nfc_reader.h"
#include "nfc_iso14443a.h"
#include "rfal_api.h"

static nfc_reader_state_t g_state = NFC_READER_IDLE;
static iso14443a_card_t   g_card;

bool nfc_reader_init(void)
{
    if (rfalInitialize() != RFAL_ERR_NONE) {
        return false;
    }

    if (rfalSetMode(RFAL_MODE_POLL_NFCA, RFAL_BR_106, RFAL_BR_106) != RFAL_ERR_
        return false;
    }

    g_state = NFC_READER_FIELD_ON;
    return true;
}

void nfc_reader_process(void)
```

RFAL's documentation states that the library supports **ISO-DEP activation/deactivation** and transceive operations in both poller and listener modes, and the X-CUBE-NFC6 sample app uses a polling loop for active and passive device detection. STMicroelectron... +1

Minimal `nfc_iso14443a.c`

C

```c
#include "nfc_iso14443a.h"
#include "rfal_nfca.h"
#include "rfal_isoDep.h"
#include <string.h>
```

```
bool iso14443a_poll(iso14443a_card_t *card)
{
    rfalNfcaSensRes sensRes;
    rfalNfcaSelRes  selRes;
    uint8_t         uid[RFAL_NFCA_CASCADE_3_UID_LEN];
    uint8_t         uidLen = 0;

    if (card == NULL) {
        return false;
    }

    memset(card, 0, sizeof(*card));

    if (rfalNfcaPollerTechnologyDetection(RFAL_COMPLIANCE_MODE_NFC, &sensRes)
        return false;
    }
```

RFAL explicitly documents support for **anticollision/collision resolution**, **card selection/activation**, and the **ISO-DEP** layer used for ISO14443-4 communication.

STMicroelectron...

Platform layer you must implement

`nfc_platform.c`

This is one of the most important files. It should contain the hardware glue for:

- SPI transfer to the ST25R3916B
- chip select control
- IRQ handling
- reset control
- timer/delay functions
- optional UART logging

ST's X-NUCLEO-NFC08A1 documentation shows that the board is driven by the host MCU through **SPI**, and on the NUCLEO-L476RG reference mapping the key lines are:

- **PA5 = SCLK**
- **PA6 = MISO**
- **PA7 = MOSI**
- **PB6 = /SS**
- **PA0 = IRQ_MCU** on one documented mapping. STMicroelectron...

Example platform helpers

```c
void nfc_cs_low(void);
void nfc_cs_high(void);
bool nfc_irq_is_asserted(void);
void nfc_reset_pulse(void);
void nfc_delay_ms(uint32_t ms);
void nfc_delay_us(uint32_t us);
bool nfc_spi_txrx(const uint8_t *tx, uint8_t *rx, uint16_t len);
```

Board pin example for NUCLEO-L476RG + X-NUCLEO-NFC08A1

From ST's user manual for the X-NUCLEO-NFC08A1, the documented host interface includes SPI and IRQ lines between the Nucleo and the board. The manual lists the reference interconnections for the NUCLEO-L476RG, including **PA5/PA6/PA7** for SPI and **PB6** for **/SS**, with IRQ available on **PA0** in one mapping table. STMicroelectron...

A simple board header could look like this:

```c
#pragma once

#define NFC_SPI_HANDLE          hspi1

#define NFC_CS_GPIO_Port        GPIOB
#define NFC_CS_Pin              GPIO_PIN_6

#define NFC_IRQ_GPIO_Port       GPIOA
#define NFC_IRQ_Pin             GPIO_PIN_0

#define NFC_RST_GPIO_Port       GPIOC
#define NFC_RST_Pin             GPIO_PIN_3   /* example only; verify your board *
```

Use the exact pinout from your chosen Nucleo and shield combination.

Suggested state-machine split

Keep the logic in three layers.

Layer 1: hardware/platform

- SPI
- GPIO
- IRQ
- timing

Layer 2: NFC protocol control

- field on/off
- technology detect
- anti-collision
- select
- activate
- transceive
- deactivate

Layer 3: application behavior

- UID whitelist
- access control
- logging
- APDU commands
- user interface

That separation makes it much easier to swap:

- STM32 model
- Nucleo board
- application logic
  without rewriting the ISO14443A flow.

Good first milestone

Build this in stages:

Stage 1

Bring up the board and confirm:

- SPI works

- IRQ fires
- RF field turns on

Stage 2

Detect a Type A card and print:

- UID
- ATQA
- SAK

Stage 3

Add ISO-DEP activation for Type 4-style cards

Stage 4

Add your application logic

This matches how ST positions the official software: start from the sample application on **X-NUCLEO-NFC08A1** and extend it. STMicroelectron... +1

Best practical advice

Instead of starting from a blank CubeIDE project, start from the **X-CUBE-NFC6 sample project** for the **X-NUCLEO-NFC08A1**, run it first, then refactor it into the structure above. ST says the package includes sample implementations for the X-NUCLEO-NFC06A1/X-NUCLEO-NFC08A1 boards on supported Nucleo boards. STMicroelectron... +1

That saves a lot of time on:

- RFAL integration
- driver configuration
- board pin setup
- timing/IRQ issues

If you want, the next best step is for me to write the **actual contents of** `nfc_platform.c` **and** `nfc_platform.h` for an STM32 HAL-based SPI implementation.

Sources

okay, please

Thought for 6s

Here is the **first file**: `nfc_platform.h`

I'm keeping it **one file at a time** so it is easier to drop into your STM32CubeIDE project.

C

```c
#ifndef NFC_PLATFORM_H
#define NFC_PLATFORM_H

#ifdef __cplusplus
extern "C" {
#endif

#include <stdbool.h>
#include <stdint.h>
#include "main.h"

/*
 * Platform abstraction layer for STM32 + ST25R3916B.
 *
 * This file exposes the hardware glue functions needed by the NFC reader
 * stack and by your application-level NFC modules.
 *
 * Keep this layer focused on:
 * - SPI transfer
 * - chip select control
 * - reset control
 * - IRQ state read
```

Notes

- This header is designed for a **HAL-based STM32CubeIDE** project.
- The GPIO and SPI macros are set as **example defaults** and should be aligned with your CubeMX configuration.
- I included both **level-based IRQ read** and **latched IRQ-event handling** so you can use either polling or interrupt-assisted flow.

Next file should be `nfc_platform.c` .

generate the datasheets for each elements in RF block

I'm basing this on the **STM32 + ST25R3916B RF section** we've been discussing, and I'm pulling the RF-chain details from ST's own docs so each block is grounded in the real hardware path.

Thought for 59s

Below is a **datasheet-style breakdown of the RF block elements** for the **STM32 + ST25R3916B reader path**, using the **X-NUCLEO-NFC08A1** RF section as the concrete reference design. The schematic shows the RF block as **ST25R3916B → EMI/filter/matching network → antenna**, with AAT parts and an external load-modulation subcircuit. STMicroelectron... +1

RF block used as reference

From ST's schematic, the RF block contains these main elements:

- **U200**: ST25R3916B NFC front end
- **Y200**: 27.12 MHz crystal
- **L300, L301**: 270 nH inductors
- **C300–C317**: fixed and variable capacitors used for matching, tuning, filtering, and AAT
- **R300, R302**: 2.4 Ω resistors
- **D300**: BAV70W dual diode
- **Q300**: NX3008NBK MOSFET
- **ANT_NFC06 / ANT1 / ANT2 / ANTC / J300**: antenna network and connection nodes.

STMicroelectron... +3

---

1) U200 — ST25R3916B NFC Reader IC

Datasheet-style summary

Item	Value
Reference	U200

Item	Value
Part number	**ST25R3916B-AQET**
Function	HF/NFC reader front end
Package	UFQFPN-32, 5 x 5 mm
Supply range	Wide supply range, documented up to **2.6–5.5 V** in production conditions
Host interface	SPI up to **10 Mbit/s**, I^2C also supported
RF support	NFC-A / ISO14443A, NFC-B / ISO14443B, NFC-F, NFC-V, card emulation, P2P
Key RF features	AFE, framing, dynamic power output, active wave shaping, noise suppression receiver, automatic antenna tuning

Role in the RF block

This is the **core RF transceiver**. It generates the 13.56 MHz drive field, receives load-modulated responses from the card, and provides the low-level framing and analog functions needed for ISO/IEC 14443 operation. ST explicitly describes it as having an **advanced analog front end** and a **highly integrated data framing system** for NFC and ISO14443 reader operation. STMicroelectron... +1

Design notes

The ST25R3916B also includes **AAT support**, **DPO**, and **AWS**, which is why you see separate AAT-related components in the RF network. STMicroelectron... +1

---

2) Y200 — RF Reference Crystal

Datasheet-style summary

Item	Value
Reference	Y200
Part number	**NDK NX2016SA-27.12MHZ-EXS00A-CS06346**

Item	Value
Frequency	**27.12 MHz**
Function	Clock reference for RF front end
Package style	SMD crystal

Role in the RF block

The ST25R3916B uses a **27.12 MHz crystal oscillator**, and ST's datasheet explicitly lists operation with a **27.12 MHz crystal**. The crystal provides the stable timing base for RF generation, modulation, demodulation, and protocol timing. STMicroelectron... +1

---

3) Antenna element — ANT_NFC06 / ANT1 / ANT2 / ANTC

Datasheet-style summary

Item	Value
References	ANT1, ANT2, ANTC, ANT_NFC06, J300
Function	Magnetic loop antenna system
Operating band	**13.56 MHz HF / NFC**
Topology	Differential antenna feed through matching/filter network
Application	ISO/IEC 14443 and NFC reader field generation

Role in the RF block

This is the **actual radiating magnetic loop structure**. AN5276 states that these ST25R39xx devices are designed to work with **magnetic loop antennas** for **13.56 MHz** applications, and the note focuses on antenna design, parameter measurement, matching, and verification. STMicroelectron...

Practical notes

Antenna performance determines:

- field strength
- read range
- interoperability
- tuning sensitivity near metal or changing environments. STMicroelectron... +1

---

4) L300, L301 — Series RF Inductors

Datasheet-style summary

Item	Value
References	L300, L301
Part number	**TDK MLJ1608WR27JT000**
Value	**270 nH**
DCR	**0.22 Ω**
Current rating	**650 mA**
Tolerance	**5%**

Role in the RF block

These inductors sit directly in the RF path between the ST25R3916B outputs and the antenna network. In this topology they are part of the **matching/filter network** that transforms the chip-side impedance to the antenna-side impedance and shapes resonance around 13.56 MHz. The schematic places them between **RFO1/RFO2** and the antenna network. STMicroelectron... +2

Design interpretation

Their exact transfer function is not spelled out line-by-line in the docs, but from the schematic position they are clearly **RF series matching/filter inductors**. That role is an engineering inference from the topology. STMicroelectron... +1

---

5) C302, C311, C314 — Variable Capacitors for AAT/Tuning

Datasheet-style summary

Item	Value
References	C302, C311, C314
Part number	**Murata LXRW0YV600-054**
Type	Variable/trimmer capacitor
Function	Antenna tuning / AAT-related tuning network

Role in the RF block

The BOM explicitly identifies these as **NFC variable capacitors**, and the schematic labels associated nets as **AAT_series** and **AAT_parallel**. These parts are used to support the antenna tuning network tied to the ST25R3916B's **AAT** capability. STMicroelectron... +2

Why they matter

They allow tuning compensation for:

- board tolerances
- antenna drift
- environmental loading
- nearby metal or housing effects. STMicroelectron... +1

---

6) Fixed Matching Capacitors — C300, C313, C301, C312, C303, C310, C305, C308, C306, C309, C315, C316, C317

These are best understood as one **capacitor bank** with different roles inside the RF network.

Datasheet-style summary

References	Value	Type
C300, C313	**180 pF**	C0G/NP0 ceramic
C301, C312	**10 pF**	C0G ceramic
C303, C310	**100 pF**	C0G ceramic

References	Value	Type
C305, C308	**680 pF**	C0G/NP0 ceramic
C306, C309	**56 pF**	MLCC
C315, C316, C317	**47 pF**	MLCC

The BOM lists these exact values and part families. STMicroelectron...

Role in the RF block

These capacitors form the **resonance, impedance matching, filtering, and AAT network** around the antenna. The schematic itself labels the whole section as **"Antenna Circuit incl. EMI Filter and Matching."** STMicroelectron... +1

Practical function by placement

This is the most honest way to describe them:

- **C300/C313 and C301/C312**: shunt/branch capacitors around the RFI paths, helping resonance and impedance shaping
- **C303/C310, C305/C308, C306/C309, C315/C316/C317**: part of the differential antenna matching/filter network and AAT-linked branch network

The exact small-signal role of each capacitor is **topology-inferred**, but the schematic and ST antenna-design documentation clearly establish that this block is the **matching and EMI-filter stage** for the NFC antenna. STMicroelectron... +2

---

7) R300, R302 — RF Damping / Series Resistors

Datasheet-style summary

Item	Value
References	R300, R302
Value	**2.4 Ω**
Package	0603
Type	Thick-film resistor

Role in the RF block

These resistors are placed in series with the antenna-side paths near **ANT1** and **ANT2**. In this position they are typically used for **damping**, **Q control**, and shaping the RF response to improve stability and EMI behavior. The schematic puts them directly in the final path into the antenna nodes. STMicroelectron... +2

Engineering note

Their purpose is not explicitly captioned as “damping” in ST’s text, so that role is an **engineering inference from location and value**.

---

8) D300 — BAV70W Dual Small-Signal Diode

Datasheet-style summary

Item	Value
Reference	D300
Part number	**BAV70W**
Type	Dual small-signal switching diode
Package	SOT-323
Voltage/current class	**100 V**, **0.17 A**, **4 ns** listed in BOM

Role in the RF block

The diode sits on the **ANT1/ANT2** side of the network. In this location it is commonly used for **clamping/protection** or shaping the external load-modulation path. The BOM confirms the exact part; the schematic shows it attached to ANT1/ANT2 with surrounding control parts. STMicroelectron... +1

Caution

The **exact functional intent** of D300 is not text-labeled in the docs. Protection/clamp/load-modulation support is a **topology-based inference**, not a directly stated ST sentence.

---

9) Q300 — NX3008NBK N-Channel MOSFET

Datasheet-style summary

Item	Value
Reference	Q300
Part number	**Nexperia NX3008NBK**
Type	N-channel trench MOSFET
Voltage/current class	BOM lists **30 V**, **400 mA** class

Role in the RF block

Q300 is connected to the **Ext_LM** node with **R303 = 47 kΩ** and tied into the antenna/diode side network. From the topology, this is very likely part of an **external load-modulation** or RF switching path. The schematic explicitly labels the net **Ext_LM**, which strongly supports that interpretation. STMicroelectron... +2

Design note

This is one of the places where the role is **mostly inferred from the schematic label and placement**, not spelled out in prose.

---

10) R301 and R303 — Support Resistors for External LM / Protection Path

Datasheet-style summary

Item	Value
R301	**100 Ω**
R303	**47 kΩ**

Role in the RF block

- **R301** sits near the diode/antenna path and likely limits or shapes current in that branch.

- **R303** biases the MOSFET gate/control node tied to **Ext_LM**.

The exact values and part types are listed in the BOM, and their location is shown in the RF schematic. STMicroelectron... +1

---

11) U.FL / Antenna connector path — P200, P202, Antenna_Con

Datasheet-style summary

Item	Value
References	P200, P202, Antenna_Con
Type	RF/antenna connection points
Connector style	U.FL connectors shown on schematic
Function	Bring RF/AAT lines to external antenna or test configuration

Role in the RF block

The X-NUCLEO schematic exposes antenna-related nodes such as **RFI1, RFI2, RFO1, RFO2, AAT_A, AAT_B, Ext_LM** to an **Antenna_Con** block, and also shows U.FL connectors on AAT lines. This supports experimentation with external antenna or tuning paths on the evaluation board. STMicroelectron... +1

---

12) Local RF decoupling around U200

Datasheet-style summary

References	Typical values
C200, C205, C207, C210, C212, C216, etc.	**1.0 µF**, **2.2 µF**, **10 nF**, **22 nF**

Role in the RF block

These capacitors are not part of the antenna matching network itself, but they are part of the **RF power integrity** around the ST25R3916B's supply domains such as **VDD_RF**,

VDD_TX, **VDD_AM**, **VDD_A**, and **VDD_D**. The schematic shows these parts directly on those rails. STMicroelectron... +1

Why they matter

Without proper local decoupling, the RF field, modulation quality, and receive performance can degrade significantly.

Functional summary of the whole RF block

Here is the cleanest system view:

Block	Main elements	Purpose
RF transceiver	U200, Y200	Generate and receive 13.56 MHz RF, framing, protocol timing
Supply integrity	C200-series decoupling caps	Stabilize RF and analog rails
Matching / resonance	L300/L301 + fixed caps	Tune antenna network to 13.56 MHz and match impedance
Automatic tuning	C302/C311/C314 + AAT paths	Compensate environment and detuning
EMI / damping	R300/R302 + filter caps	Control Q, improve stability/EMI
Antenna path	ANT1/ANT2/ANTC/J300	Create magnetic field for ISO14443 communication
External LM/protection	D300, Q300, R301, R303	Auxiliary clamp/load-modulation branch

This breakdown matches ST's RF schematic organization and its antenna-design documentation. STMicroelectron... +2

Important caution

I can state with confidence:

- the **exact parts and values**

- the **overall RF topology**
- that the block is an **antenna circuit including EMI filter and matching**
- that the ST25R3916B supports **AAT** and is the NFC front end. STMicroelectron... +2

For a few passives, especially **D300/Q300/R301/R303**, the **precise circuit purpose** is partly inferred from the schematic rather than explicitly narrated by ST.

If you want, I can turn this into a **formal document-style datasheet pack**, one component sheet per page, in the format:
Description / Key Specs / Function / Connection / Design Notes / Replacement Criteria.

Sources